DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Discover. Learn. Empower.

Experiment No.: 10

Student Name: Probal Dhali
UID: 24BAI71013
Branch: AIT-CSE
Section/Group: B
Semester: 5TH
Date of Performance:
Subject Name: Competitive Coding – II
Subject Code: 24CSP-305

Experiment No: 10.1

Aim: To write a C++ program to count the number of islands in a 2D grid.

Objective: To understand graph traversal using Depth-First Search (DFS) to identify connected components in a grid.

Problem Statement: Given an m × n grid consisting of '1' (land) and '0' (water), count the number of islands. An island is formed by connecting adjacent land cells horizontally or vertically.

LeetCode Link:

Procedure/Algorithm/Pseudocode:

Code:

```
#include <bits/stdc++.h>

using namespace std;

class Solution {

public:

   void dfs(vector<vector<char>>& grid, int i, int j) {

      int m = grid.size();

      int n = grid[0].size();

      if (i < 0 || j < 0 || i >= m || j >= n ||

         grid[i][j] == '0') {

         return;

      }

      grid[i][j] = '0';

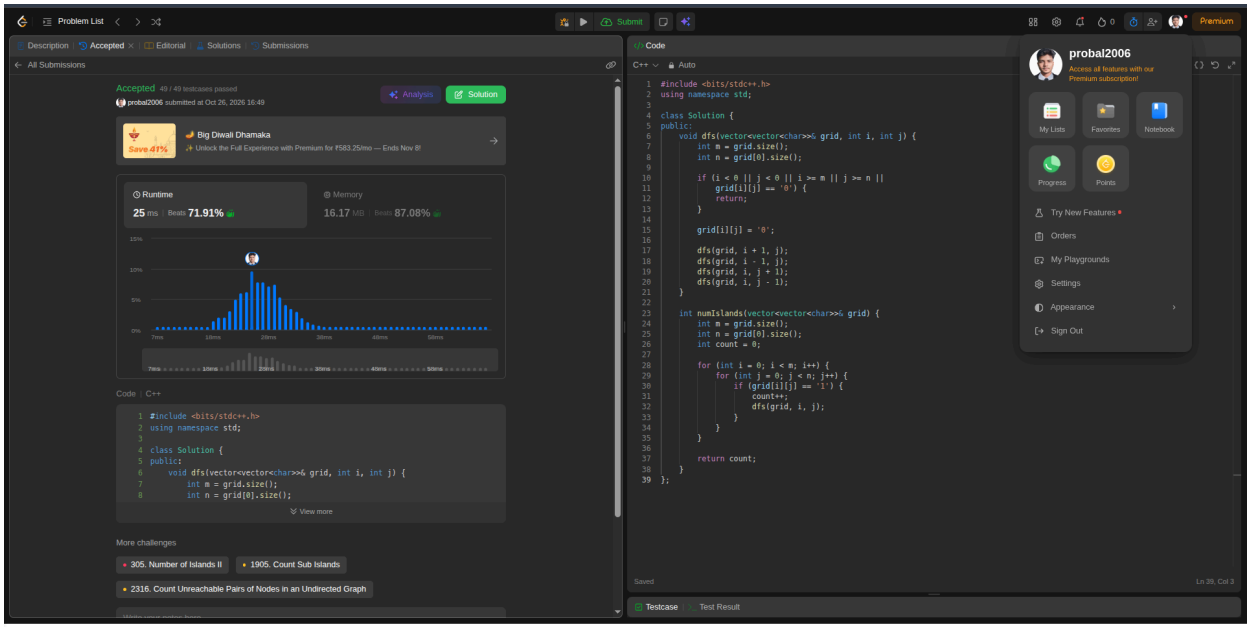
      dfs(grid, i + 1, j);

      dfs(grid, i - 1, j);

      dfs(grid, i, j + 1);
```

```
        dfs(grid, i, j - 1);
    }
    int numIslands(vector<vector<char>>& grid) {
        int m = grid.size();
        int n = grid[0].size();
        int count = 0;
        for (int i = 0; i < m; i++) {
            for (int j = 0; j < n; j++) {
                if (grid[i][j] == '1') {
                    count++;
                    dfs(grid, i, j);
                }
            }
        }
        return count;
    }
};
```

Output:



Observations/Outcome: The program successfully counts the number of islands using DFS by visiting and marking all connected land cells.

Time Complexity: $O(m \times n)$

Space Complexity: $O(m \times n)$

Learning outcomes :

- Understand graph traversal using DFS.
- Identify connected components in a 2D grid.
- Apply recursion to solve grid-based problems.
- Analyze time and space complexity.

Experiment No.: 10.2

Aim: To write a C++ program to determine whether a valid path exists between two vertices in an undirected graph.

Objective: To understand graph representation and traversal using Depth-First Search (DFS).

Problem Statement: Given an undirected graph with n vertices and a list of edges, determine whether a valid path exists between a source vertex and a destination vertex.

LeetCode Link:

Procedure/Algorithm/Pseudocode:

1. Create an adjacency list to represent the graph.
2. Add each edge to the adjacency list in both directions.
3. Start DFS from the source vertex.
4. Mark each visited vertex.
5. If the destination is reached, return true.
6. If all reachable vertices are explored without reaching the destination, return false.

Code:

```
#include <bits/stdc++.h>

using namespace std;

class Solution {

public:

   bool validPath(int n, vector<vector<int>>& edges,

                  int source, int destination) {

      vector<vector<int>> adj(n);

      for (auto& edge : edges) {

         adj[edge[0]].push_back(edge[1]);

         adj[edge[1]].push_back(edge[0]);

      }

      vector<bool> visited(n, false);

      stack<int> st;

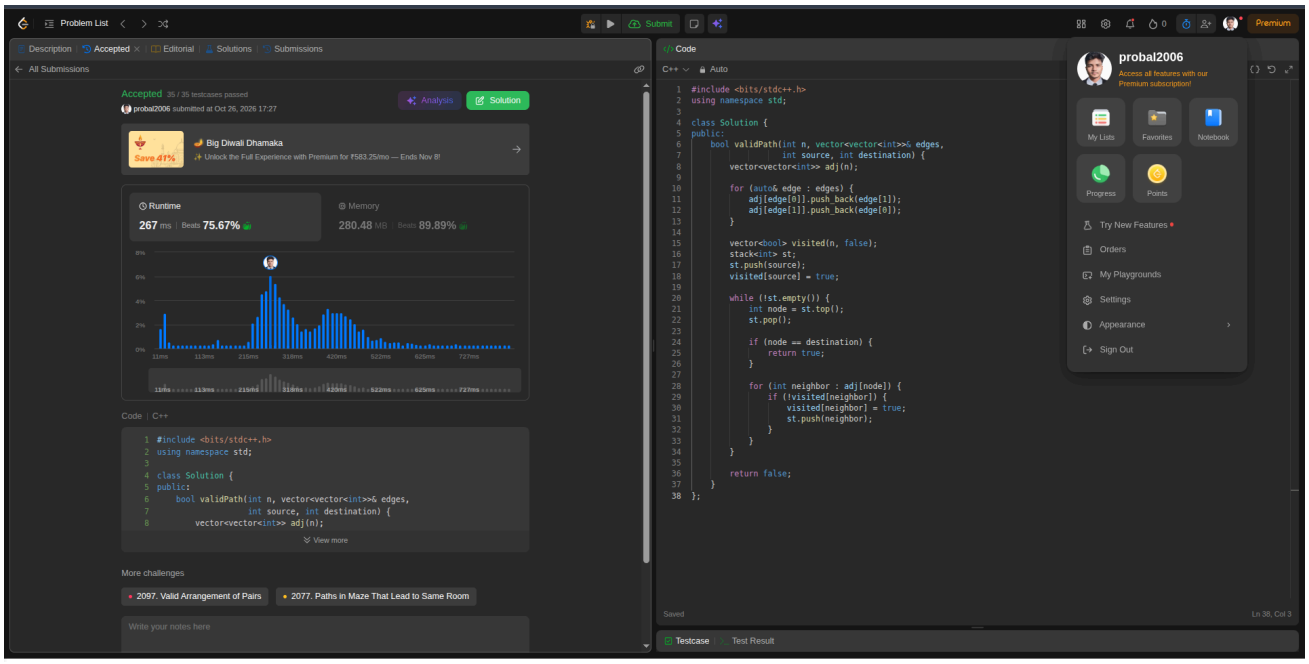
      st.push(source);

      visited[source] = true;

      while (!st.empty()) {
```

```
            int node = st.top();
            st.pop();
            if (node == destination) {
                return true;
            }
            for (int neighbor : adj[node]) {
                if (!visited[neighbor]) {
                    visited[neighbor] = true;
                    st.push(neighbor);
                }
            }
        }
        return false;
    }
};
```

Output:



Observations/Outcome: The program successfully determines whether a path exists between the source and destination vertices using DFS.

Time Complexity: $O(V + E)$

Space Complexity: $O(V + E)$

Learning outcomes :

- Understand graph representation using adjacency lists.
- Implement DFS using a stack.
- Determine connectivity between two vertices.
- Analyze the time and space complexity of graph traversal.